MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR
ET DE LA RECHERCHE SCIENTIFIQUE

UNIVERSITÉ TUNIS EL MANAR
Institut Supérieur d'Informatique — ISI

CAHIER DES CHARGES

Système Intelligent de Gestion des Accès Physiques

Avec Reconnaissance Faciale, IoT, IA, Cloud et Mobile

Guide complet d'installation, d'utilisation et d'exemples

MODULE	IOT
DÉPARTEMENT	GLSI — Institut Supérieur d'Informatique
NIVEAU	1ère Ingénieur 1
ANNÉE	2025 – 2026

RÉALISÉ PAR
Arij Belmabrouk
Wassim Lourimi
Yasmine Hsayri

Table des matières

PRÉSENTATION DU PROJET **3**
1.1 Titre du projet . . . 3
1.2 Description générale . . . 3
1.3 Cadre du projet . . . 3

CONTEXTE ET PROBLÉMATIQUE **3**
2.1 Contexte . . . 3
2.2 Limites des systèmes existants . . . 3
2.3 Solution proposée . . . 4

OBJECTIFS DU PROJET **5**
3.1 Objectif principal . . . 5
3.2 Objectifs spécifiques . . . 5
3.3 Résultats attendus . . . 5

PÉRIMÈTRE ET ACTEURS **5**
4.1 Périmètre du projet . . . 5
4.2 Acteurs du système . . . 6

BESOINS FONCTIONNELS **6**
5.1 Module IoT — Système Embarqué . . . 6
5.1.1 Contrôle d'accès par reconnaissance faciale . . . 6
5.1.2 Surveillance environnementale . . . 6
5.1.3 Logique de sécurité incendie [FONCTIONNALITÉ CLÉ] . . . 6
5.2 Module Cloud — Backend . . . 6
5.2.1 Gestion des utilisateurs . . . 7
5.2.2 Gestion des accès et logs . . . 7
5.2.3 Gestion des données environnementales . . . 7
5.3 Module Mobile — Application . . . 7
5.3.1 Authentification . . . 7
5.3.2 Tableau de bord . . . 7
5.3.3 Gestion des employés . . . 7
5.3.4 Alertes et notifications . . . 7
5.3.5 Historique des accès . . . 7

BESOINS NON FONCTIONNELS **8**

ARCHITECTURE TECHNIQUE **8**
7.1 Vue d'ensemble . . . 8
7.2 Flux de données principal . . . 8

MATÉRIEL ET COMPOSANTS **10**

STACK TECHNOLOGIQUE **11**

SCÉNARIOS D'UTILISATION **11**

PLANIFICATION **13**

LIVRABLES ATTENDUS **13**

CONTRAINTES ET RISQUES **13**
13.1 Contraintes techniques . . . 13
13.2 Contraintes de projet . . . 14
13.3 Analyse des risques . . . 14

PRÉSENTATION DU PROJET

1.1 Titre du projet

Système Intelligent de Gestion des Accès Physiques avec Reconnaissance Faciale, IoT, Intelligence Artificielle, Cloud et Application Mobile.

1.2 Description générale

Ce projet consiste à concevoir et développer un système automatisé de contrôle d'accès physique à une porte sécurisée. Le système combine plusieurs technologies modernes : la reconnaissance faciale en temps réel pour identifier les personnes autorisées, des capteurs IoT pour surveiller l'environnement (température, gaz), un backend hébergé dans le cloud pour centraliser les données, et une application mobile permettant à l'administrateur de gérer le système à distance.

Fonctionnalité critique : en cas de détection d'incendie ou de gaz dangereux, la porte reste ouverte automatiquement pour permettre l'évacuation, contrairement au comportement normal où elle se ferme après chaque accès.

1.3 Cadre du projet

Paramètre	Valeur
Type	Projet académique (cours annuel)
Équipe	Arij Belmabrouk / Wassim Lourimi / Yasmine Hsayri
Durée estimée	2 mois
Domaines couverts	IoT, IA, Mobile, Cloud
Matériel principal	ESP32

CONTEXTE ET PROBLÉMATIQUE

2.1 Contexte

La sécurité des accès physiques est un enjeu majeur dans de nombreux environnements : bâtiments administratifs, entreprises, laboratoires, etc. Les systèmes traditionnels de contrôle d'accès (badges magnétiques, codes PIN, empreintes digitales) présentent plusieurs limites qui réduisent leur fiabilité et leur sécurité.

2.2 Limites des systèmes existants

- **Badges et cartes :** peuvent être volés, perdus ou dupliqués facilement.
- **Codes PIN :** partagés ou oubliés, ne garantissent pas l'identité réelle.
- **Empreintes digitales :** peuvent être copiées, et difficiles à modifier en cas de compromission.
- **Absence de surveillance environnementale :** aucune réaction automatique en cas

de danger (incendie, gaz).

- **Traçabilité manuelle :** suivi des accès peu fiable, difficile à auditer.
- **Gestion réactive :** les incidents sont détectés après coup, pas en temps réel.

2.3 Solution proposée

Notre solution intègre :

- La **reconnaissance faciale par IA** pour une authentification biométrique infalsifiable.
- Des **capteurs IoT** pour surveiller en permanence l'environnement.
- Un **cloud backend** pour centraliser, stocker et gérer les données.
- Une **application mobile** pour administrer le système à distance et recevoir des alertes en temps réel.
- Une **logique de sécurité incendie** : porte maintenue ouverte automatiquement si danger détecté.

OBJECTIFS DU PROJET

3.1 Objectif principal

Développer un système autonome et intelligent de gestion des accès physiques, capable d'identifier les personnes autorisées par reconnaissance faciale, de surveiller l'environnement en temps réel, et d'être piloté depuis une application mobile connectée au cloud.

3.2 Objectifs spécifiques

- **Implémenter un module de reconnaissance faciale en temps réel sur ESP32.**
- **Intégrer des capteurs IoT (température, gaz, mouvement) pour la surveillance environnementale.**
- **Développer une logique de sécurité incendie (ouverture automatique de la porte en cas de danger).**
- **Créer un backend cloud pour la gestion des utilisateurs et la centralisation des logs.**
- **Développer une application mobile multiplateforme pour l'administration à distance.**
- **Assurer la traçabilité complète de tous les événements d'accès.**
- **Envoyer des notifications push en temps réel en cas d'événement critique.**

3.3 Résultats attendus

Résultat	Description
Porte intelligente	Ouverture automatique pour visages autorisés, fermeture après passage
Sécurité incendie	Porte ouverte si température > seuil ou gaz détecté
Application mobile	Gestion des accès, historique, alertes push
Backend cloud	API REST, base de données, stockage des photos
Alertes temps réel	Notification si inconnu détecté ou danger environnemental
Historique complet	Logs de tous les accès accessibles depuis le mobile

PÉRIMÈTRE ET ACTEURS

4.1 Périmètre du projet

Le projet couvre les composants suivants :

- Système embarqué (ESP32) avec caméra et capteurs
- Module de reconnaissance faciale (IA locale)
- Communication IoT via protocole MQTT
- Backend cloud (API REST + base de données)

- Application mobile Android/iOS
- Système de notifications push

4.2 Acteurs du système

Acteur	Type	Rôle
Administrateur	Humain	Gère des accès des employés, consulte les logs, reçoit les alertes via mobile
Employé / Utilisateur	Humain	Se présente devant la caméra pour accéder
Inconnu	Humain	Tentative d'accès non autorisée — alerte déclenchée
ESP32	Système	Exécute la reconnaissance faciale et contrôle la porte
Capteurs IoT	Matériel	Collectent données température, gaz, mouvement
Application Mobile	Logiciel	Interface d'administration distante
Backend Cloud	Logiciel	Centralise données, authentification, logs

BESOINS FONCTIONNELS

5.1 Module IoT — Système Embarqué

5.1.1 Contrôle d'accès par reconnaissance faciale

- Le système détecte automatiquement la présence d'une personne via le capteur PIR.
- La caméra Raspberry Pi V2 capture une image du visage.
- Le module IA compare le visage aux profils enregistrés.
- Si le visage est reconnu → le moteur est activé, la porte s'ouvre.
- Le capteur PIR détecte le passage → la porte se referme automatiquement.
- Si aucun passage dans les 30 secondes → fermeture automatique.
- Si le visage est inconnu → accès refusé, photo sauvegardée, alerte envoyée.

5.1.2 Surveillance environnementale

- Le capteur DHT22 mesure la température en continu.
- Le capteur MQ-135 détecte la présence de gaz (CO, NH3, benzène, etc.).
- Les données sont publiées via MQTT vers le broker Mosquitto.
- Le backend souscrit aux topics MQTT et stocke les valeurs dans le cloud.

5.1.3 Logique de sécurité incendie [FONCTIONNALITÉ CLÉ]

- Si température > seuil critique (ex. 45°C) → porte maintenue **OUVERTE** automatiquement.
- Si gaz dangereux détecté → porte maintenue **OUVERTE** automatiquement.
- La porte se referme uniquement lorsque les conditions redeviennent normales.
- Une alerte push est envoyée immédiatement à l'administrateur.

5.2 Module Cloud — Backend

5.2.1 Gestion des utilisateurs

- Ajouter un nouvel employé (nom, prénom, email, photo).
- Modifier les informations d'un employé existant.
- Supprimer un employé (révocation d'accès automatique).
- Consulter la liste complète des employés avec filtrage.

5.2.2 Gestion des accès et logs

- Enregistrer chaque tentative d'accès (autorisée ou refusée) avec horodatage.
- Stocker la photo capturée lors de chaque tentative.
- Fournir une API REST pour consulter l'historique des accès.
- Permettre le filtrage par date, employé, statut (succès/échec).

5.2.3 Gestion des données environnementales

- Recevoir et stocker les données MQTT (température, gaz).
- Détecter les dépassements de seuils critiques.
- Déclencher des notifications push via Firebase Cloud Messaging.

5.3 Module Mobile — Application

5.3.1 Authentification

- L'administrateur se connecte avec email et mot de passe.
- Gestion de session sécurisée avec tokens JWT.
- Réinitialisation du mot de passe par email.

5.3.2 Tableau de bord

- Affichage en temps réel de la température et du niveau de gaz.
- Indicateur de statut de la porte (ouverte/fermée).
- Liste des derniers accès avec photos et horodatages.

5.3.3 Gestion des employés

- Ajouter, modifier, supprimer un employé depuis le mobile.
- Uploader la photo de l'employé depuis la galerie ou l'appareil photo.
- Recherche et filtrage dynamique de la liste.

5.3.4 Alertes et notifications

- Recevoir des notifications push si un inconnu tente d'accéder.
- Recevoir des notifications si température ou gaz dépassent les seuils.
- Historique des alertes consultable dans l'application.

5.3.5 Historique des accès

- Consulter l'historique complet des accès avec filtrage.
- Voir la photo capturée pour chaque tentative.
- Exporter ou partager les logs si nécessaire.

BESOINS NON FONCTIONNELS

Critère	Exigence	Détail
Performance	Reconnaissance < 2 sec	Traitement local sur ESP32 pour minimiser la latence
Disponibilité	Système autonome 24/7	Fonctionnement même en cas de coupure internet partielle
Sécurité	Communications chiffrées	HTTPS/TLS pour toutes les API, tokens JWT pour l'authentification
Fiabilité	Capteurs stables	Lectures régulières avec gestion des erreurs et valeurs aberrantes
Scalabilité	Architecture extensible	Backend cloud capable d'accueillir plusieurs points d'accès
Utilisabilité	Interface intuitive	Application mobile simple, responsive, en français
Traçabilité	Logs complets	Chaque événement enregistré avec horodatage précis et photo
Temps réel	Alertes instantanées	Délai de notification push $<$ 5 secondes après détection

ARCHITECTURE TECHNIQUE

7.1 Vue d'ensemble

L'architecture du système est composée de quatre couches interconnectées :

Couche	Composants	Rôle
Couche IoT (Edge)	ESP32 + Capteurs + Caméra + Moteur	Acquisition, traitement local, contrôle physique
Couche Communication	MQTT (Mosquitto) + HTTPS	Transmission des données entre les composants
Couche Cloud	Backend Flask + PostgreSQL + Firebase	Stockage, logique métier, notifications
Couche Mobile	Application Flutter	Interface utilisateur, administration, alertes

7.2 Flux de données principal

Scénario accès autorisé :

1. Capteur PIR détecte une présence → déclenche la caméra
2. Caméra capture l'image → OpenCV détecte et encode le visage
3. Comparaison avec la base locale → identité trouvée
4. Relais activé → Le moteur ouvre la porte
5. Capteur PIR interne détecte le passage → fermeture après délai
6. Événement JSON généré → envoyé au backend cloud via API REST
7. Backend stocke le log + notifie l'app mobile

Scénario incendie / gaz :

1. DHT22 détecte température > seuil critique **OU** MQ-135 détecte gaz
2. ESP32 publie alerte sur topic MQTT
3. ESP32 force l'ouverture de la porte (relais maintenu actif)
4. Backend reçoit l'alerte MQTT → déclenche notification Firebase
5. App mobile reçoit notification push urgente
6. Porte reste ouverte jusqu'au retour à la normale

MATÉRIEL ET COMPOSANTS

Composant	Modèle	Rôle dans le projet
Ordinateur embarqué	NodeMCU ESP32-3S	Cerveau du système : IA, contrôle IoT, communication
Caméra	OV2640 2MP camera	Capture des images pour la reconnaissance faciale
Capteur température	DHT22	Surveillance thermique — détection chaleur anormale
Capteur gaz	MQ-135	Détection CO, NH3, benzène — alerte incendie/gaz
Capteur mouvement	PIR HC-SR501	Détection présence et confirmation de passage
Contrôle porte	28BYJ-48 Stepper Motor / SG90 Micro Servo	Verrouillage/déverrouillage physique de la porte
Interface relais	ULN2003 Driver Board	Permet au ESP32 de contrôler le moteur
Smartphone	Android / iOS	Exécution de l'application mobile administrateur

STACK TECHNOLOGIQUE

Couche	Technologie	Justification
Langage embarqué	Arduino	Bibliothèques IA/IoT riches, simple sur ESP32
Vision par ordinateur	OpenCV	Détection de visages en temps réel, open source
Reconnaissance faciale	Modèle IA de vision par ordinateur	Encodage et comparaison de visages précis
Protocole IoT	MQTT (Mosquitto broker)	Léger, faible latence, idéal pour IoT
Backend	Flask REST Framework	Robuste, sécurisé
Base de données	PostgreSQL	Relationnelle, fiable, open source
Hébergement cloud	Render	Déploiement backend accessible depuis mobile
Stockage fichiers	Render	Stockage des photos d'accès
Application mobile	Flutter	Multiplateforme (Android + iOS), performant
Notifications push	Firebase Cloud Messaging	Standard industrie pour notifications mobiles
Authentification	JWT (JSON Web Tokens)	Sécurisé, stateless, adapté aux APIs REST

SCÉNARIOS D'UTILISATION

Scénario 1 — Accès autorisé
Scénario 2 — Accès refusé (inconnu)
Scénario 3 — Détection de danger (incendie / gaz)
Scénario 4 — Administration depuis le mobile

Scénario 1 — Accès autorisé

Étape	Action
1	Un employé s'approche de la porte
2	Le capteur PIR détecte sa présence et déclenche la caméra
3	Le système reconnaît le visage en moins de 2 secondes
4	La porte s'ouvre automatiquement
5	L'employé entre, le PIR interne détecte le passage
6	La porte se referme après 3 secondes
7	L'événement est enregistré dans le cloud et visible dans l'app mobile

Scénario 2 — Accès refusé (inconnu)

Étape	Action
1	Une personne non enregistrée se présente devant la porte
2	Le système tente la reconnaissance → échec
3	La porte reste fermée
4	La photo de l'inconnu est capturée et envoyée au cloud
5	L'administrateur reçoit une notification push urgente sur son mobile
6	L'événement est journalisé avec statut *Inconnu* + photo

Scénario 3 — Détection de danger (incendie / gaz)

Étape	Action
1	Le capteur DHT22 détecte une température de 50°C (seuil dépassé)
2	OU le capteur MQ-135 détecte une concentration anormale de gaz
3	Le ESP32 force l'ouverture de la porte immédiatement
4	La porte reste ouverte (relais maintenu actif)
5	Une alerte critique est publiée via MQTT vers le cloud
6	L'administrateur reçoit une notification push d'urgence
7	Quand les conditions redeviennent normales, la porte reprend son fonctionnement standard

Scénario 4 — Administration depuis le mobile

Étape	Action
1	L'administrateur ouvre l'application mobile et se connecte
2	Il consulte le tableau de bord (température, gaz, statut porte)
3	Il ajoute un nouvel employé avec sa photo
4	Le profil est synchronisé avec la ESP32 via le cloud
5	Il consulte l'historique des accès filtrés par date ou employé

PLANIFICATION

Phase	Contenu	Durée estimée
Phase 1 — Analyse & Conception	Rédaction CDC, diagrammes UML, choix technologiques	2 semaines
Phase 2 — Setup IoT	Configuration ESP32, connexion capteurs, tests matériels	2 semaines
Phase 3 — Module IA	Implémentation reconnaissance faciale, tests précision	3 semaines
Phase 4 — Contrôle porte	Relais + moteur, logique PIR, scénario incendie	2 semaines
Phase 5 — Backend Cloud	API REST, base de données, hébergement, MQTT broker	3 semaines
Phase 6 — App Mobile	Développement Flutter, authentification, notifications push	4 semaines
Phase 7 — Intégration	Connexion de tous les modules, tests end-to-end	2 semaines
Phase 8 — Tests & Documentation	Tests complets, correction bugs, rédaction rapport	2 semaines

LIVRABLES ATTENDUS

Livrable	Description	Format
Système embarqué fonctionnel	ESP configuré avec tous les capteurs et la caméra	Matériel physique
Module reconnaissance faciale	Scripts Python, modèle entraîné, tests de performance	Code source
Backend cloud déployé	API REST documentée, base de données, hébergement actif	URL + code source
Application mobile	APK Android fonctionnel (+ iOS si possible)	Fichier installable
Rapport technique	Architecture, choix techniques, résultats tests	Document PDF
Vidéo de présentation	Présentation de l'objectif et de l'approche + démonstration live pratique	Présentation + fichier MP4
Code source complet	Tout le code versionné sur GitHub	Repository Git

CONTRAINTES ET RISQUES

13.1 Contraintes techniques

- Puissance de calcul limitée du ESP32 pour la reconnaissance faciale en temps réel.

- Dépendance à la qualité de l'éclairage pour la reconnaissance faciale.
- Nécessité d'une connexion réseau stable pour la synchronisation cloud.
- Le DHT22 n'est pas un vrai détecteur d'incendie (simulation académique).

13.2 Contraintes de projet

- Durée limitée imposant un périmètre fonctionnel réaliste.
- Budget matériel à définir (ESP32, capteurs, Moteur).
- Coordination entre les membres de l'équipe sur les différentes couches.

13.3 Analyse des risques

Risque	Probabilité	Impact	Mitigation
Reconnaissance faciale lente sur ESP32	Moyenne	Fort	Optimiser le code, réduire la résolution d'image
Faux positifs / négatifs IA	Faible	Fort	Augmenter le dataset d'entraînement, ajuster le seuil
Latence réseau cloud	Moyenne	Moyen	Traitement local prioritaire, cloud en asynchrone
Panne capteur DHT22/MQ-135	Faible	Fort	Gestion des erreurs avec valeurs par défaut + alertes
Disponibilité serveur cloud	Faible	Moyen	Choisir un hébergeur fiable avec uptime garanti
Retard développement mobile	Moyenne	Moyen	Commencer avec interface web simple en fallback